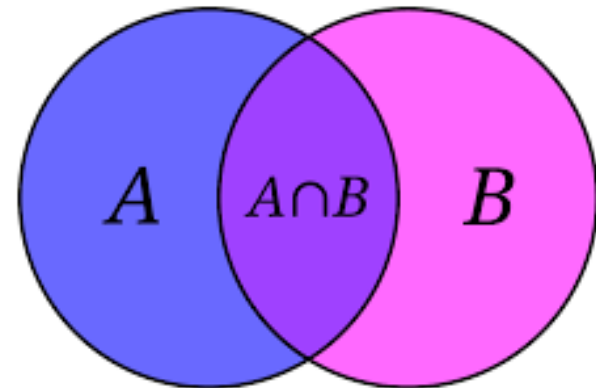
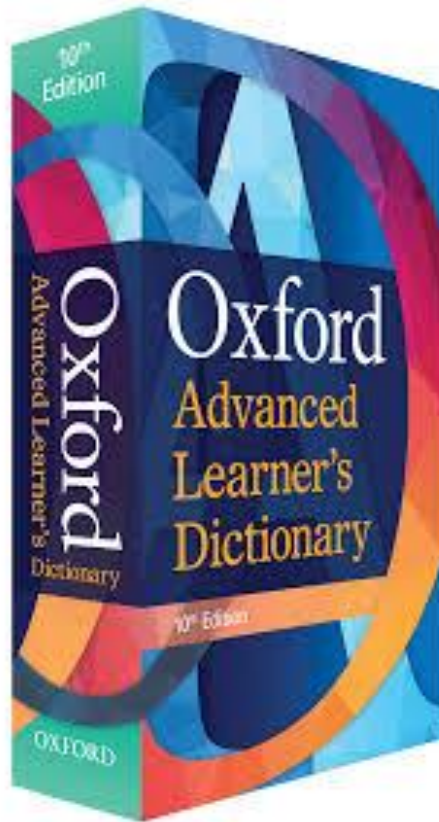
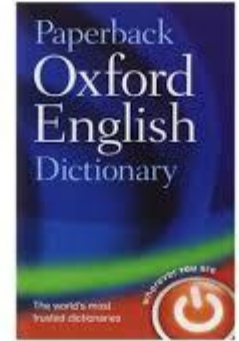


Dictionaries & Sets



Dictionaries



- **Dictionary: object that stores a collection of data**
 - Each element consists of a *key* and a *value*
 - Often referred to as *mapping* of key to value
 - **Key must be an immutable object**
 - To retrieve a specific value, use the key associated with it
 - Format for creating a dictionary
dictionary = {*key1:val1*, *key2:val2*}

Dictionaries:

Format for creating a dictionary

```
>>> phonebook = {'Chris':'555-1111',  
'Katie':'555-2222', 'Joanne':'555-3333'} ←
```

```
>>> phonebook ←
```

```
{'Chris': '555-1111', 'Joanne': '555-3333',  
'Katie':'555-2222'}
```

```
>>>
```

What happened to the
display order?



Retrieving a Value from a Dictionary

- As of Python 3.7, dictionaries are ordered
- General format for retrieving value from dictionary: *dictionary[key]*
 - If `key` in the dictionary, associated value is returned, otherwise, `KeyError` exception is raised

Dictionaries: retrieve a specific value

```
1 >>> phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'} ↵
```

```
2 >>> phonebook['Chris'] ↵
```

```
3 '555-1111'
```

```
4 >>> phonebook['Joanne'] ↵
```

```
5 '555-3333'
```

```
6 >>> phonebook['katie'] ↵
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>KeyError: 'katie'

```
>>>
```

Retrieving a Value from a Dictionary

- **Test whether a key is in a dictionary using the `in` and `not in` operators**
 - Helps prevent `KeyError` exceptions

1 >>> phonebook = {'Chris': '555-1111', 'Katie': '555-2222', 'Joanne': '555-3333'} ↵

2 >>> if 'Chris' in phonebook: ↵

3 print(phonebook['Chris']) ↵↵

4

5 555-1111

6 >>>

```
>>> if 'Chris' in phonebook:
...     print(phonebook['Chris'])
...
555-1111
```

```
>>> if 'Chri' in phonebook:
...     print(phonebook['Chris'])
...
>>>
```

Retrieving a Value from a Dictionary

- **Test whether a key is in a dictionary using the `in` and `not in` operators**
 - Helps prevent `KeyError` exceptions

```
1 >>> phonebook = {'Chris':'555-1111', 'Katie':'555-2222'} ↵
```

```
2 >>> if 'Joanne' not in phonebook: ↵
```

```
3     print('Joanne is not found ') ↵↵
```

```
4
```

```
5 Joanne is not found.
```

```
6 >>>
```

Some Dictionary Methods

- **get method**: gets a value associated with specified key from the dictionary
 - Format: `dictionary.get(key, default)`
 - `default` is returned if `key` is not found
 - Default value is `None`
 - Alternative to `[]` operator
 - Cannot raise `KeyError` exception

Some Dictionary Methods

- get method: example

1 >>> phonebook = {'Chris':'555-1111', 'Katie':'555-2222'} ↵

2 >>> value = phonebook.get('Katie', 'Entry not found') ↵

3 >>> print(value) ↵

4 555-2222

value = phonebook['Katie']

5 >>> value = phonebook.get('Andy', 'Entry not found') ↵

6 >>> print(value) ↵

7 Entry not found

8 >>> phonebook['Chris']

9 '555-1111'

10 >>>

Adding Elements to an Existing Dictionary

- Dictionaries are mutable objects
- To add a new key-value pair:

dictionary[key] = value

- If key exists in the dictionary, the value associated with it will be changed

```
1 >>> phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'} ↵
2 >>> phonebook['Joe'] = '555-0123' ↵
3 >>> phonebook['Chris'] = '555-4444' ↵
4 >>> phonebook ↵
5 {'Chris': '555-4444', 'Katie': '555-2222', 'Joanne': '555-3333', 'Joe': '555-0123'}
6 >>>
```

Creating an Empty Dictionary

- **Two methods to create an empty dictionary:**
 1. Use `{ }` → e.g., `pb={ }` ↵
 2. Use built-in function `dict()` → `pb=dict()` ↵
- Elements can be added to the dictionary as program executes

Creating an Empty Dictionary– Example #4

```
1 >>> phonebook = { } ↵
```

```
2 >>> phonebook['Chris'] = '555-1111' ↵
```

```
3 >>> phonebook['Katie'] = '555-2222' ↵
```

```
4 >>> phonebook['Joanne'] = '555-3333' ↵
```

```
5 >>> phonebook ↵
```

```
6 {'Chris': '555-1111', 'Katie': '555-2222', 'Joanne': '555-3333'}
```

```
7 >>>
```

Deleting Elements From an Existing Dictionary

- To delete a key-value pair:

```
del dictionary[key]
```

- If key is not in the dictionary, `KeyError` exception is raised

Deleting Elements From an Existing Dictionary Example

```
1 >>> phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'} ↵
2 >>> phonebook ↵
3 {'Chris': '555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'}
4 >>> del phonebook['Chris'] ↵
5 >>> phonebook ↵
6 {'Katie': '555-2222', 'Joanne': '555-3333'}
7 >>> del phonebook['Chris'] ↵
8 Traceback (most recent call last):
9   File "<stdin>", line 1, in <module>
10  KeyError: 'Chris'
11 >>>
```

Deleting Elements From an Existing Dictionary Example

- To prevent a `KeyError` exception from being raised, you should use the *in* operator to determine whether a key exists before you try to delete it and its associated value

```
1 >>> phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'} ↵
```

```
2 >>> if 'Chris' in phonebook: ↵
```

```
3     del phonebook['Chris'] ↵ ↵
```

```
4
```

```
5 >>> phonebook ↵
```

```
6 {'Katie':'555-2222', 'Joanne':'555-3333'}
```

```
7 >>>
```

Some Dictionary Methods (cont'd.)

- **pop method**: returns value associated with specified key and removes that key-value pair from the dictionary
 - Format: `dictionary.pop(key, default)`
 - `default` is returned if `key` is not found
 - Raise `KeyError`, if `key` not present and `default` value not specified.

Some Dictionary Methods

(pop method Example)

```
1 >>> phonebook = {'Chris':'555-1111', ↵  
2                  'Katie':'555-2222', ↵  
3                  'Joanne':'555-3333'} ↵  
4 >>> phone_num = phonebook.pop('Chris', 'Entry not found') ↵  
5 >>> phone_num ↵  
6 '555-1111'  
7 >>> phonebook ↵  
8 {'Katie': '555-2222', 'Joanne': '555-3333'}  
9 >>> phone_num = phonebook.pop('Andy', 'Element not found') ↵  
10 >>> phone_num ↵  
11 'Element not found'  
12 >>> phonebook ↵  
13 {'Katie': '555-2222', 'Joanne': '555-3333'}  
14 >>>
```

'Chris was removed
from the dictionary'



Some Dictionary Methods

- **clear method:** deletes all the elements in a dictionary, leaving it empty
 - Format: `dictionary.clear()`

```
1 >>> phonebook = {'Chris':'555-1111', 'Katie':'555-2222'} ↵
```

```
2 >>> phonebook ↵
```

```
3 {'Chris': '555-1111', 'Katie': '555-2222'} ↵
```

```
4 >>> phonebook.clear() ↵
```

```
5 >>> phonebook ↵
```

```
6 {} ↵
```

```
7 >>>
```

Getting the Number of Elements and Mixing Data Types

- **len function:** used to obtain number of elements in a dictionary
- **Although, keys must be immutable objects, the associated values can be any type of object**
 - One dictionary can include keys of several different immutable types
- **Values stored in a single dictionary can be of different types**

Getting the Number of Elements and Mixing Data Types – Example #2

1 >>> mixed_up = {'abc':1, 999:'yada yada', (3, 6, 9):[3, 6, 9]} ←

2 >>> mixed_up←

3 {'abc': 1, 999: 'yada yada', (3, 6, 9): [3, 6, 9]}

4 >>>

- The 1st element is 'abc':1.
 - Its key is the string 'abc' and the value is the integer 1.
- The 2nd element is 999:'yada yada'.
 - Its key is integer 999 and the value is the string 'yada yada'.
- The 3rd element is (3, 6, 9):[3, 6, 9].
 - The key is tuple (3, 6, 9) and the value is the list [3, 6, 9].

Remember: keys
are immutable

Using for Loop to Iterate Over a Dictionary

- **Use a for loop to iterate over a dictionary**
 - General format: `for key in dictionary:`

`for var in dictionary:`
`statement`
`statement`
`etc.`

Using for Loop to Iterate Over a Dictionary

```
1 >>> phonebook = {'Chris':'555-1111', ↵  
2                  'Katie':'555-2222', ↵  
3                  'Joanne':'555-3333'} ↵  
4 >>> for key in phonebook: ↵  
5 ...     print(key) ↵  
6 ... ↵  
7  
8 Chris  
9 Katie  
10 Joanne  
11 >>> for key in phonebook: ↵  
12 ...     print(key, phonebook[key]) ↵  
13 ... ↵  
14  
15 Chris 555-1111  
16 Katie 555-2222  
17 Joanne 555-3333  
18 >>>
```

Some Dictionary Methods (cont'd.)

- **items method:** returns all the dictionaries keys and associated values
 - Format: `dictionary.items()`
 - Returned as a *dictionary view*
 - Each element in dictionary view is a **tuple** which contains a key and its associated value

Some Dictionary Methods (cont'd.)

- items method: example

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'}
```

If we call the `phonebook.items()` method, it returns the following sequence:

```
dict_items([('Chris', '555-1111'), ('Katie', '555-2222'), ('Joanne', '555-3333')])
```



```
1 d = {"a": 1, "b": 2, "c": 3}
2
3 print(d)
4 it_d = d.items()
5 print(it_d)
6 print(type(it_d))
7 lst_d = list(it_d)
8 print(lst_d)
9 print(type(lst_d))
```

```
{'a': 1, 'b': 2, 'c': 3}
dict_items([('a', 1), ('b', 2), ('c', 3)])
<class 'dict_items'>
[('a', 1), ('b', 2), ('c', 3)]
<class 'list'>
```

Some Dictionary Methods (cont'd.)

- items method: returns all the dictionaries keys and associated values
 - Use a `for` loop to iterate over the tuples in the sequence
 - Can use a variable which receives a tuple, or can use two variables which receive key and value

Some Dictionary Methods (*for Example.*)

```
1 >>> phonebook = {'Chris':'555-1111', ↵  
2                  'Katie':'555-2222', ↵  
3                  'Joanne':'555-3333'} ↵  
4 >>> for key, value in phonebook.items(): ↵  
5 ...     print(key, value) ↵  
6 ... ↵  
7 Chris 555-1111  
8 Katie 555-2222  
9 Joanne 555-3333  
10 >>>
```

Some Dictionary Methods (cont'd.)

- **keys method:** returns all the dictionaries keys as a sequence
 - Format: *dictionary.keys()*

phonebook = {'Chris':'555-1111', 'Katie':'555-2222', 'Joanne':'555-3333'}

If we call the `keys` method, it will return the following sequence:

```
dict_keys(['Chris', 'Katie', 'Joanne'])
```

```
1 d = {"a": 1, "b": 2, "c": 3}
2
3 print(d)
4 k_d = d.keys()
5 print(k_d)
6 print(type(k_d))
7 lst_d = list(k_d)
8 print(lst_d)
9 print(type(lst_d))
```

```
{'a': 1, 'b': 2, 'c': 3}
dict_keys(['a', 'b', 'c'])
<class 'dict_keys'>
['a', 'b', 'c']
<class 'list'>
```

Some Dictionary Methods (cont'd.)

- **keys method:** use of a for loop to iterate over the sequence that is returned from the **keys** method

```
1 >>> phonebook = {'Chris':'555-1111', ↵  
2                  'Katie':'555-2222', ↵  
3                  'Joanne':'555-3333'} ↵  
4 >>> for key in phonebook.keys(): ↵  
5 ...     print(key) ↵  
6 ... ↵  
7 Chris  
8 Katie  
9 Joanne  
10 >>>
```

Some Dictionary Methods (cont'd.)

- values method: returns all the dictionaries values as a sequence

- Format: *dictionary.values()*

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222',  
             'Joanne':'555-3333'}
```

If we call the phonebook.**values()** method, it returns the following sequence:

```
dict_values(['555-1111', '555-2222', '555-3333'])
```

```
1 d = {"a": 1, "b": 2, "c": 3}
2
3 print(d)
4 v_d = d.values()
5 print(v_d)
6 print(type(v_d))
7 lst_d = list(v_d)
8 print(lst_d)
9 print(type(lst_d))
-
```

```
{'a': 1, 'b': 2, 'c': 3}
dict_values([1, 2, 3])
<class 'dict_values'>
[1, 2, 3]
<class 'list'>
```


Some Dictionary Methods (cont'd.)

- values method: using a for loop

```
1 >>> phonebook = {'Chris':'555-1111', ↵  
2                  'Katie':'555-2222', ↵  
3                  'Joanne':'555-3333'} ↵  
4 >>> for val in phonebook.values(): ↵  
5 ...     print(val) ↵  
6 ... ↵  
7 555-1111  
8 555-2222  
9 555-3333  
10 >>>
```

Some Dictionary Methods (cont'd.)

Table 9-1 Some of the dictionary methods

Method	Description
<code>clear</code>	Clears the contents of a dictionary.
<code>get</code>	Gets the value associated with a specified key. If the key is not found, the method does not raise an exception. Instead, it returns a default value.
<code>items</code>	Returns all the keys in a dictionary and their associated values as a sequence of tuples.
<code>keys</code>	Returns all the keys in a dictionary as a sequence of tuples.
<code>pop</code>	Returns the value associated with a specified key and removes that key-value pair from the dictionary. If the key is not found, the method returns a default value.
<code>popitem</code>	Returns a randomly selected key-value pair as a tuple from the dictionary and removes that key-value pair from the dictionary.
<code>values</code>	Returns all the values in the dictionary as a sequence of tuples.

Nested Dictionaries

```
nestedDict = { keyA: { $k_1$ :  $v_1$ , ...,  $k_m$ :  $v_m$ }, keyB: { $k_1$ :  $v_1$ , ...,  $k_n$ :  $v_n$ }}
```

```
citizens = { 111: {'name': 'Kevin', 'gender': 'Female'},  
            222: {'name': 'Smith', 'gender': 'Male'} }
```

Accessing elements of a nested dictionary

```
1 >>> print(citizens[111]['name']) ←
```

```
2 Kevin
```

```
3 >>> print(citizens[222]['gender']) ←
```

```
4 Male
```

```
5 >>>
```

Nested Dictionaries (cont')

```
citizens = { 111: {'name': 'Kevin', 'gender': 'Female'},  
            222: {'name': ' Smith', 'gender': 'Male'} }
```

Adding elements of a nested dictionary

```
1 >>> citizens ↵
```

```
2 {111: {'name': 'Kevin', 'gender': 'Female'}, 222: {'name': ' Smith', 'gender': 'Male'}}
```

```
3 >>> citizens[333]={} ↵
```

```
4 >>> citizens ↵
```

```
5 {111: {'name': 'Kevin', 'gender': 'Female'}, 222: {'name': ' Smith', 'gender': 'Male'},  
  333: {}}
```

```
6 >>>
```

Nested Dictionaries (cont')

```
citizens = { 111: {'name': 'Kevin', 'gender': 'Female'},  
            222: {'name': 'Smith', 'gender': 'Male'} }
```

Adding elements of a nested dictionary

```
1 >>>
```

```
2 >>> citizens[333]['name'] = 'Guy' ↵
```

```
3 >>> citizens[333]['gender'] = 'Male' ↵
```

```
4 >>> citizens[333]['height'] = '188' ↵
```

```
5 >>>
```

```
6 >>> print(citizens[333]) ↵
```

```
7 {'name': 'Guy', 'gender': 'Male', 'height': '188'}↵
```

```
8 >>>
```

Nested Dictionaries

```
citizens = { 111: {'name': 'Kevin', 'gender': 'Female'},  
            222: {'name': 'Smith', 'gender': 'Male'},  
            333: {'name': 'Guy', 'gender': 'Male', 'height': '188'} }
```

Adding another dictionary to the nested dictionary

```
1 >>> citizens[444] = {'name': 'Dana', 'gender': 'Female'} ←
```

```
2 >>>
```

```
3 >>> print(citizens[444]) ←
```

```
4 {'name': 'Dana', 'gender': 'Female'}
```

```
5 >>>
```

Nested Dictionaries

```
citizens = { 111: {'name': 'Kevin', 'gender': 'Female'},  
            222: {'name': ' Smith', 'gender': 'Male'},  
            333: {'name': ' Guy', 'gender': 'Male', 'height': '188'},  
            444: {'name': 'Dana', 'gender': 'Female'} }
```

Deleting elements from a nested dictionary

```
1 >>> del citizens[333]['height'] ↵
```

```
2 >>>
```

```
3 >>> print(citizens[333]) ↵
```

```
4 {'name': ' Guy', 'gender': 'Male'}
```

```
5 >>>
```

Nested Dictionaries

```
citizens = { 111: {'name': 'Kevin', 'gender': 'Female'},  
            222: {'name': ' Smith', 'gender': 'Male'},  
            333: {'name': ' Guy', 'gender': 'Male'},  
            444: {'name': 'Dana', 'gender': 'Female'} }
```

Deleting a dictionary from a nested dictionary

```
1 >>> del citizens[333], citizens[444] ↵
```

```
2 >>> print(citizens) ↵
```

```
3 { 111: {'name': 'Kevin', 'gender': 'Female'}, 222: {'name': ' Smith', 'gender':  
'Male'}}
```

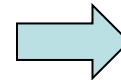
```
4 >>>
```


Nested Dictionaries

```
citizens = { 111: {'name': 'Kevin', 'gender': 'Female'},  
            222: {'name': 'Smith', 'gender': 'Male'} }
```

Iterating through a nested dictionary

```
1 >>> for citId, citInfo in citizens.items(): ↵  
2 ...     print("\nCitizen ID:", citId) ↵  
3 ...     for key in citInfo:  
4 ...         print(key+":",citInfo[key]) ↵  
5 ...
```



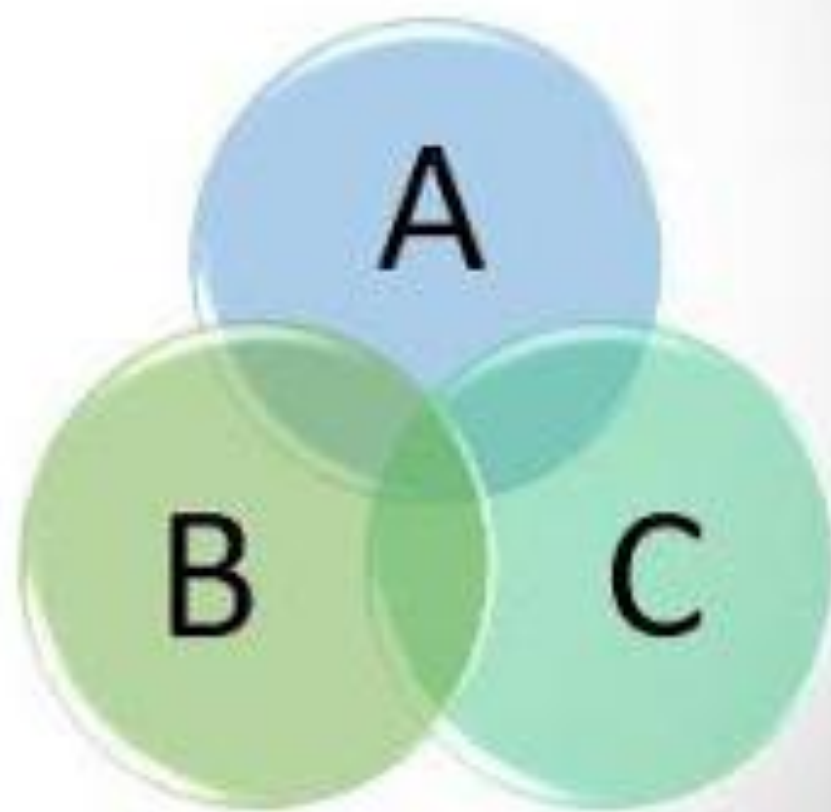
```
Citizen ID: 111  
name: Kevin  
gender: Female
```

```
Citizen ID: 222  
name: Smith  
gender: Male  
>>>
```

```
>>> for citId, citInfo in citizens.items():  
...     print("\nCitizen ID:", citId)  
...     for key in citInfo:  
...         print(key+":",citInfo[key])  
...  
  
Citizen ID: 111  
name: Kevin  
sex: Female  
  
Citizen ID: 222  
name: Smith  
sex: Male
```



Python Sets



Sets

- **Set: object that stores a collection of data in same way as mathematical set**
 - All items must be unique
 - Set is unordered
 - Elements can be of different **immutable** data types

Creating a Set

- **set function: used to create a set**
 - For empty set, call `set()`
 - `myset = set()`
 - For non-empty set, call `set(argument)` where *argument* is an object that contains iterable elements

Creating a Set

- set function: used to create a set
 - For non-empty set, call `set(argument)` where *argument* is an object that contains iterable elements
 - e.g., *argument* can be a list, string, or tuple
 - `myset = set(['a', 'b', 'c'])`
 - If *argument* is a string, each character becomes a set element

```
1 >>> myset = set('abc') ↵
```

```
2 >>> myset ↵
```

```
3 {'b', 'a', 'c'}
```

Creating a Set

- set function: used to create a set
 - If *argument* contains duplicates, only one of the duplicates will appear in the set
 - `myset = set('aaabc')`
 - After this statement executes, the `myset` variable will reference a set containing the elements 'a', 'b', and 'c'

```
1 >>> myset = set('aaabc') ↵
```

```
2 >>> myset ↵
```

```
3 {'b', 'a', 'c'}
```

Creating a Set

- set function: used to create a set
 - If *argument* is a string, each character becomes a set element

The following does not accomplish the task either:

This does not do what we intend.

```
myset = set('one two three') ↵
```

```
myset ↵
```

```
{'h', 'w', 't', 'e', ' ', 'n', 'o', 'r'}
```

Creating a Set

- set function: used to create a set
 - If *argument* is a string, each character becomes a set element
 - For set of strings, pass them to the function as a list

OK, this works.

```
myset = set(['one', 'two', 'three']) ↵
```

```
myset ↵
```

```
{'two', 'one', 'three'}
```


Getting the Number of Elements

- len function: returns the number of elements in the set
- Sets are mutable objects

1 >>> myset = set([1, 2, 3, 4, 5]) ↵

2 >>> len(myset) ↵

3 5

4 >>> myset = {1, 2, 3, 4, 5} ↵

✗ 5 >>> myset = set([1, [2, 3], 4, 5]) ↵

Adding Elements

- **add method**: adds an element to a set

```
1 >>> myset = set() ↵
2 >>> myset.add(1) ↵
3 >>> myset.add(2) ↵
4 >>> myset.add(3) ↵
5 >>> myset ↵
6 {1, 2, 3}
7 >>> myset.add(2) ↵
8 >>> myset
9 {1, 2, 3}
10 >>>
```

Note:

myset.add("abc")

מוסיף את כל המילה "abc" בשלמותה
לקבוצה כאיבר אחד.

Updating Elements

- **update method**: adds a group of elements to an existing set
 - Argument must be a sequence containing iterable elements, and each of the elements is added to the set

```
1 >>> myset = set([1, 2, 3]) ↵
```

```
2 >>> myset.update([4, 5, 6]) ↵
```

```
3 >>> myset ↵
```

```
4 {1, 2, 3, 4, 5, 6}
```

```
5 >>>
```

Updating Elements

- update method: adds a group of elements to an existing set
- Another example:

```
1 >>> set1 = set([1, 2, 3]) ↵  
2 >>> set2 = set([8, 9, 10]) ↵  
3 >>> set1.update(set2) ↵  
4 >>> set1 ↵  
5 {1, 2, 3, 8, 9, 10}  
6 >>> set2 ↵  
7 {8, 9, 10}  
8 >>>
```

Updating Elements

- update method: adds a group of elements to an existing set

- 3rd example:

```
1 >>> myset = set([1, 2, 3]) ↵
```

```
2 >>> myset.update('abc') ↵
```

```
3 >>> myset ↵
```

```
4 {1, 2, 3, 'a', 'c', 'b'}
```

```
5 >>>
```

Deleting Elements From a Set

- **remove and discard methods:** remove the specified item from the set
 - The item that should be removed is passed to both methods as an argument
 - Behave differently when the specified item is not found in the set
 - `remove` method raises a `KeyError` exception
 - `discard` method does not raise an exception
- **clear method:** clears all the elements of the set

Deleting Elements From a Set

```
1 >>> myset = set([1, 2, 3, 4, 5]) ↵
2 >>> myset ↵
3 {1, 2, 3, 4, 5}
4 >>> myset.remove(1) ↵
5 >>> myset ↵
6 {2, 3, 4, 5}
7 >>> myset.discard(5) ↵
8 >>> myset ↵
9 {2, 3, 4}
10 >>> myset.discard(99) ↵
11 >>>
12 >>> myset ↵
13 {2, 3, 4}
14 >>> myset.remove(99) ↵
15 Traceback (most recent call last):
16   File "<stdin>", line 1, in <module>
17 KeyError: 99
```

Deleting Elements From a Set

- **clear method: clears all the elements of the set**

```
1 >>> myset = set([1, 2, 3, 4, 5]) ↵
```

```
2 >>> myset ↵
```

```
3 {1, 2, 3, 4, 5}
```

```
4 >>> myset.clear() ↵
```

```
5 >>> myset ↵
```

```
6 set()
```

```
7 >>>
```


Using the for Loop With a Set

- **A for loop can be used to iterate over elements in a set**
 - General format: `for item in set:`
 - The loop iterates once for each element in the set
- for var in set:
 statement
 statement
 etc.

Using the for Loop With a Set

- **A for loop example :**

```
1 >>> myset = set(['a', 'b', 'c']) ↵
```

```
2 >>> for val in myset: ↵
```

```
3 ...     print(val) ↵
```

```
4 ... ↵
```

```
5 a
```

```
6 c
```

```
7 b
```

```
8 >>>
```

Using the `in`, and `not in` Operators With a Set

- The `in` operator can be used to test whether a value exists in a set

```
1 >>> myset = set([1, 2, 3]) ↵
```

```
2 >>> if 1 in myset: ↵
```

```
3 ...     print('The value 1 is in the set.') ↵
```

```
4 ... ↵
```

```
5 The value 1 is in the set.
```

```
6 >>>
```

Using the `in`, and `not in` Operators With a Set

- Similarly, the ***not in*** operator can be used to test whether a value does not exist in a set

```
1 >>> myset = set([1, 2, 3]) ↵
```

```
2 >>> if 99 not in myset: ↵
```

```
3 ...     print('The value 99 is not in the set.') ↵
```

```
4 ... ↵
```

```
5 The value 99 is not in the set.
```

```
6 >>>
```

pop Removes Some Item

- **The returned value of the pop() method is the removed item**
- **We don't know what item will be removed in advance**

```
>>> months={"January", "February", "March"}  
>>>months.pop()  
'January'  
>>>months  
{'February', 'March'}
```



del Completely Deletes the Set

- **The syntax is *del set***

```
>>> months={"January", "February", "March"} ↵
```

```
>>>del months ↵
```

```
>>>months ↵
```

Traceback (most recent call last):

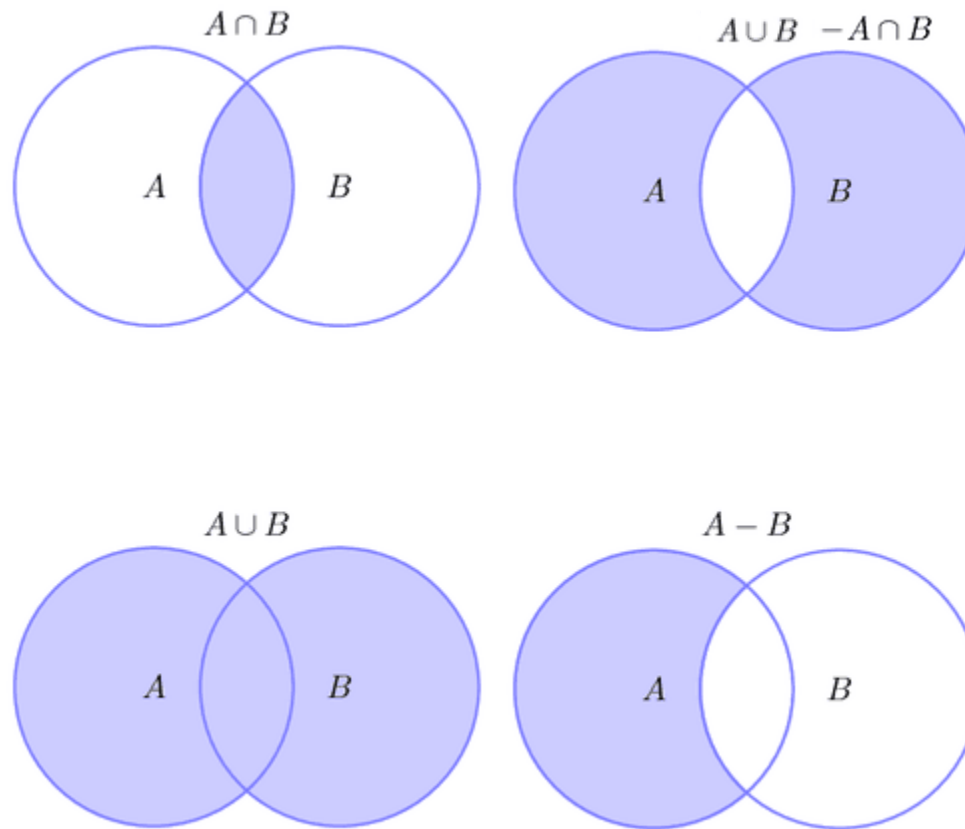
File "<stdin>", line 1, in <module>

NameError: name 'months' is not defined

```
>>>
```



Python Sets Methods



Finding the *union* of Sets

- **Union of two sets**: a set that contains all the elements of both sets
- **To find the union of two sets:**
 - Use the `union` method
 - Format: `set1.union(set2)`

```
1 >>> set1 = set([1, 2, 3, 4]) ↵
2 >>> set2 = set([3, 4, 5, 6]) ↵
3 >>> set3 = set1.union(set2) ↵
4 >>> set3 ↵
5 {1, 2, 3, 4, 5, 6}
6 >>>
```

Duplicates
are removed

Finding the *union* of Sets

- **Another way to find union of two sets:**

- Use the `|` operator

- Format: `set1 | set2`

```
1 >>> set1 = set([1, 2, 3, 4]) ↵
```

```
2 >>> set2 = set([3, 4, 5, 6]) ↵
```

```
3 >>> set3 = (set1 | set2) ↵
```

```
4 >>> set3 ↵
```

```
5 {1, 2, 3, 4, 5, 6}
```

```
6 >>>
```

- Both techniques return a new set which contains the union of both sets



Finding the *intersection* of Sets

- **Intersection of two sets**: a set that contains only the elements found in both sets
- **To find the intersection of two sets:**
 1. Use the `intersection` method
 - Format: `set1.intersection(set2)`
 2. Use the `&` operator
 - Format: `set1 & set2`
- Both techniques return a new set which contains the intersection of both sets

Finding the *intersection* of Sets

- **Examples of Intersection of two sets**

```
1 >>> set1 = set([1, 2, 3, 4]) ↵  
2 >>> set2 = set([3, 4, 5, 6]) ↵  
3 >>> set3 = set1.intersection(set2) ↵  
4 >>> set3 ↵  
5 {3, 4}  
6 >>>
```

```
1 >>> set1 = set([1, 2, 3, 4]) ↵  
2 >>> set2 = set([3, 4, 5, 6]) ↵  
3 >>> set3 = set1 & set2 ↵  
4 >>> set3 ↵  
5 {3, 4}  
6 >>>
```

- Both techniques return a new set which contains the intersection of both sets



Finding the *difference* of Sets

- **Difference of two sets**: a set that contains the elements that appear in the first set but do not appear in the second set
- **To find the difference of two sets:**
 1. Use the `difference` method
 - Format: `set1.difference(set2)`
 2. Use the `-` operator
 - Format: `set1 - set2`

Finding the *difference* of Sets

- **Example of Difference of two sets**

```
1 >>> set1 = set([1, 2, 3, 4]) ↵  
2 >>> set2 = set([3, 4, 5, 6]) ↵  
3 >>> set3 = set1.difference(set2) ↵  
4 >>> set3 ↵  
5 {1, 2}  
6 >>>
```

```
1 >>> set1 = set([1, 2, 3, 4]) ↵  
2 >>> set2 = set([3, 4, 5, 6]) ↵  
3 >>> set3 = set1 - set2 ↵  
4 >>> set3 ↵  
5 {1, 2}  
6 >>>
```

- Both techniques return a new set which contains the difference of both sets



Finding the *symmetric difference* of Sets

- **Symmetric difference of two sets**: a set that contains the elements that are not shared by the two sets
- **To find the symmetric difference of two sets:**
 1. Use the `symmetric_difference` method
 - Format: `set1.symmetric_difference(set2)`
 2. Use the `^` operator
 - Format: `set1 ^ set2`

Finding the *symmetric difference* of Sets

- **Example of Symmetric Difference of sets**

```
1 >>> set1 = set([1, 2, 3, 4]) ↵
```

```
2 >>> set2 = set([3, 4, 5, 6]) ↵
```

```
3 >>> set3 = set1.symmetric_difference(set2) ↵
```

```
4 >>> set3 ↵
```

```
5 {1, 2, 5, 6}
```

```
6 >>>
```

```
1 >>> set1 = set([1, 2, 3, 4]) ↵
```

```
2 >>> set2 = set([3, 4, 5, 6]) ↵
```

```
3 >>> set3 = set1 ^ set2 ↵
```

```
4 >>> set3 ↵
```

```
5 {1, 2, 5, 6}
```

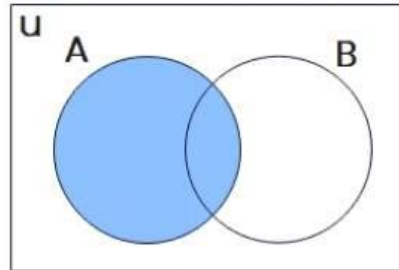
```
6 >>>
```

- Both techniques return a new set which contains the difference of both sets

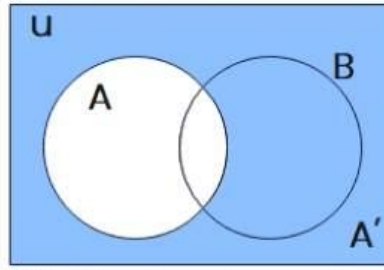


Python Sets Methods

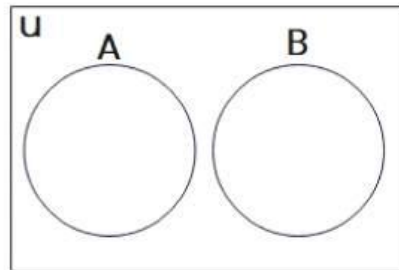
Operations of Sets and Venn Diagrams



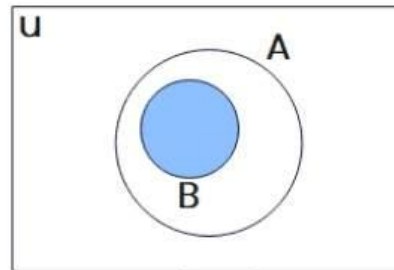
Set A



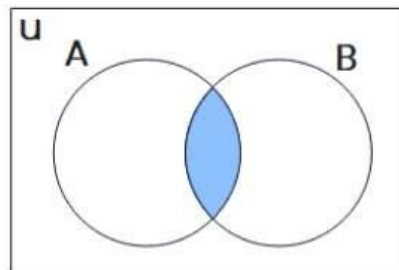
A' is the Complement of A



A & B are disjoint sets

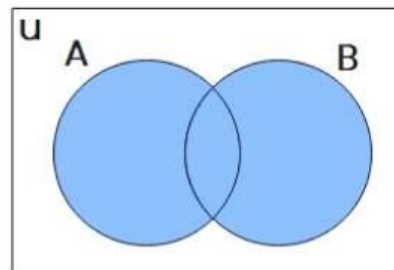


$B \subset A$
B is the proper subset of A



$A \cap B$

A & B are overlapping sets



$A \cup B$

A & B are overlapping sets

Finding *subsets* and *supersets*

- **Set A is subset of set B if all the elements in set A are included in set B**
- **To determine whether set A is subset of set B:**
 1. Use the `issubset` method
 - Format: `setA.issubset(setB)`
 2. Use the `<=` operator
 - Format: `setA <= setB`

Finding *subsets* and *supersets* (cont'd.)

- **Set A is superset of set B if it contains all the elements of set B**
- **To determine whether set A is superset of set B**
 - Use the `issuperset` method
 - Format: `setA.issuperset(setB)`
 - Use the `>=` operator
 - Format: `setA >= setB`

Finding *subsets* and *supersets*

Examples

```
1 >>> set1 = set([1, 2, 3, 4]) ↵
```

```
2 >>> set2 = set([2, 3]) ↵
```

```
3 >>> set2.issubset(set1) ↵
```

```
4 True
```

```
5 >>>
```

```
6 >>> set1.issuperset(set2) ↵
```

```
7 True
```

```
8 >>>
```

```
9 >>> set2 <= set1 ↵
```

```
10 True
```

```
11 >>>
```

```
12 >>> set1 >= set2 ↵
```

```
13 True
```

```
14 >>>
```

```
15 >>> set1 <= set2 ↵
```

```
16 False
```

```
17 >>>
```



isdisjoint Method

- **Returns True if two sets are disjoint sets**

```
>>> months1={"January", "February", "March"} ↵
```

```
>>> months2={"April", "May", "June"} ↵
```

```
>>> months3={" July", "August", "June"} ↵
```

```
>>> months1.isdisjoint(months2) ↵
```

```
True
```

```
>>>
```

```
>>> months2.isdisjoint(months3) ↵
```

```
False
```

```
>>>
```



set() Constructor

- The *set()* constructor converts to set

```
>>> m=("Jan", "Feb", "Mar") ↵
```

```
>>> m ↵
```

```
('Jan', 'Feb', 'Mar')
```

```
>>> type(m) ↵
```

```
<class 'tuple'>
```

```
>>>
```

```
>>> m=set(("Jan","Feb","Mar")) ↵
```

```
>>> m ↵
```

```
{'Jan', 'Feb', 'Mar'}
```

```
>>> type(m) ↵
```

```
<class 'set'>
```

```
>>>
```